

Практикум 2.2 — «Геометрия и градиентный спуск»

Методические указания для студентов

Цели

- Освоить геометрический смысл скалярного произведения и линейных преобразований.
- Реализовать градиентный спуск для 1D и 2D функций; оценить влияние шага обучения.
- Использовать TensorFlow GradientTape для вычисления градиента.

Задания

1) Вычислите скалярное произведение двух 2D-векторов; определите угол между ними. Реализуйте матрицу поворота на 30° и проверьте результат.

2) Реализуйте градиентный спуск для $f(x)=(x-3)^2$. Постройте лог/график сходимости. Исследуйте влияние `learning_rate`.

3) Реализуйте градиентный спуск для $f(w_1, w_2)=(w_1-1)^2+2(w_2+2)^2$. Опишите, какая координата сходится быстрее и почему.

4) Повторите пункт 2/3 с использованием `tf.GradientTape` и сравните градиенты.

Критерии оценивания

Критерий	Макс. балл	Отметки преподавателя
Геометрические вычисления (dot, угол, поворот)	20	
Градиентный спуск 1D: корректность и анализ шага	25	
Градиентный спуск 2D: корректность и объяснение	35	
Оформление ноутбука и выводы	20	

Шаблон отчёта (кратко)

1. Цель работы.
2. Код и промежуточные результаты.
3. График/лог сходимости.
4. Анализ ошибок и выводы.

Практикум 2.2 — Геометрия и градиентный спуск

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
```

Задание 1: Геометрия (dot, угол, поворот)

```
In [ ]: a = np.array([1., 0.])
b = np.array([0.5, np.sqrt(3)/2])
dot = np.dot(a, b)
cos_theta = dot / (np.linalg.norm(a) * np.linalg.norm(b))
theta = np.degrees(np.arccos(np.clip(cos_theta, -1, 1)))
phi = np.radians(30)
R = np.array([[np.cos(phi), -np.sin(phi)], [np.sin(phi), np.cos(phi)]])
a_rot = R @ a
print(f'dot={dot:.3f}, theta≈{theta:.1f}°')
print('a_rot:', a_rot)
```

Задание 2: Градиентный спуск 1D

```
In [ ]: def f(x):
    return (x-3)**2

x = 8.0
lr = 0.2
xs, fs = [], []
for step in range(25):
    grad = 2*(x-3)
    x -= lr*grad
    xs.append(x)
    fs.append(f(x))
    print(f'{step:02d}: x={x:.4f}, f(x)={f(x):.6f}')
plt.figure(); plt.plot(fs); plt.title('Сходимость f(x)'); plt.xlabel('step'); plt.y
```

Задание 3: Градиентный спуск 2D

```
In [ ]: def f2(w):
    return (w[0]-1)**2 + 2*(w[1]+2)**2

w = np.array([3.0, -6.0])
lr = 0.1
vals = []
for step in range(30):
    grad = np.array([2*(w[0]-1), 4*(w[1]+2)])
```

```
w -= lr * grad
vals.append(f2(w))
print(f'{step:02d}: w={w}, f(w)={f2(w):.6f}')
plt.figure(); plt.plot(vals); plt.title('Сходимость f(w)'); plt.xlabel('step'); plt
```

(Опционально) TensorFlow GradientTape

```
In [ ]: # Если доступен TensorFlow, раскомментируйте:
# import tensorflow as tf
# x = tf.Variable(5.0)
# with tf.GradientTape() as tape:
#     y = (x-3.0)**2
# dy_dx = tape.gradient(y, x)
# print('dy/dx:', dy_dx.numpy())
```

Выводы

Опишите влияние шага обучения и различия между 1D и 2D задачами.